

GraphReduce Feature Families

A practical guide to what each family computes, when to use it, and how the families fit together

Technical report for the Kurve RSC benchmark

August 16, 2026

Contents

1	Executive summary	2
2	The mental model: annotate, reduce, then join	2
2.1	One running example	3
3	The seven families	3
3.1	base: schema-aware rollups	3
3.2	semantic: caller-defined meaning	4
3.3	conditional: composition within time windows	4
3.4	temporal: numeric magnitude by lookback window	5
3.5	sequence: cadence, concentration, and active span	5
3.6	episode: row volume versus distinct records	6
3.7	context: peer-relative row values	7
4	How the families work together	7
5	Configuration example	7
6	Time correctness and window semantics	9
7	Budgeting and family selection	9
7.1	Recommended staged rollout	10
8	Common pitfalls	10
9	Quick decision guide	10
10	Source basis	11

1 Executive summary

GraphReduce turns related tables into a model-ready table at a chosen grain. A typical graph might reduce many **events** rows to one row per **user**, then join those user-level summaries into the parent table. Feature families are reusable sets of SQL operations that decide *what information survives that reduction*.

GraphReduce 1.10.1 recognizes seven SQL feature-family names:

Family	The question it answers	Typical output
base	How much, how many, and how recently?	Type-aware aggregates, category/text summaries, recency, window counts
semantic	What domain concept does this row represent?	Caller-defined predicate or value annotations
conditional	What kind of activity occurred in each window?	Conditional count, share, presence, and change
temporal	How did numeric magnitude vary by lookback window?	Windowed sum, average, minimum, and maximum
sequence	Was activity steady, concentrated, or bursty?	Rate, recent share, burst ratio, active span
episode	How many rows versus distinct real-world records?	Row counts and distinct-primary-key counts
context	How does a row compare with its peers?	Peer-group size and difference from peer mean

The practical default is **base**. Add families only where their signal has a clear interpretation: **temporal** for measurements, **sequence** for cadence, **conditional** for status or category mix, **episode** for duplicate-prone joins, **semantic** for known business rules, and **context** for a defensible peer group.

Scope. The seven named families described here are implemented by the SQL auto-feature planner (`sql_auto_features`) and are used through `do_transformations_sql()`. GraphReduce also has automatic aggregation paths for pandas, Dask, Spark, and Daft, but those paths do not implement this same seven-family program.

2 The mental model: annotate, reduce, then join

Suppose the desired output is one row per user and each user has many events:

```

users (parent grain)          events (child grain)
+-----+                    +-----+-----+-----+-----+
| user_id | 1 <----- | event_id | user_id | status | amount |
+-----+          many  +-----+-----+-----+-----+

```

With `reduce=True`, GraphReduce processes the child relation before joining it to the parent:

```

raw child rows
-> point-in-time filtering
-> row annotations (`semantic` and `context`)
-> grouped feature calculation by the edge key
-> one reduced row per parent key
-> left join into the parent

```

This ordering explains two important ideas:

1. A feature family describes relational operations at a node and graph hop, not a fixed list of model columns.
2. **semantic** and **context** first enrich individual child rows. The normal reduction then summarizes those enriched values to the parent grain.

The graph itself must be configured with `auto_features=True`, and SQL graphs must be executed with `do_transformations_sql()`.

2.1 One running example

The examples below use a cut date of **2024-01-10** and three lookback periods: 1, 7, and 30 days. All rows belong to user 1.

event_id	event_time	status	amount
101	2024-01-01	invited	10
102	2024-01-05	paid	20
103	2024-01-09	invited	30

At the cut date, the 1-day window contains one event, the 7-day window contains two, and the 30-day window contains all three. GraphReduce applies its feature window before reduction, so rows after the reference time do not contribute.

3 The seven families

3.1 base: schema-aware rollups

3.1.1 What it does

base is the broad baseline. GraphReduce samples the relation, infers physical and semantic types, and chooses aggregations that are valid for each type.

- Numeric columns use the graph’s configured type/function map, commonly yielding sum, average, minimum, and maximum.
- Boolean columns are summed as 0/1 values, which gives a count of true rows.
- Identifier-like columns contribute one general count instead of a redundant count for every identifier.
- Categorical columns get a distinct count. Selected values also get count, share, and presence features. Low-cardinality columns use all sampled values; high-cardinality columns use the most frequent `categorical_top_k` values and an `other` bucket.
- Date and timestamp columns can receive min/max-style summaries according to the configured type map.
- Text-looking columns can receive inexpensive SQL shape features when `auto_text_features=True`: character and word-length summaries plus count/share/presence for empty text, URLs, numbers, question marks, and exclamation marks.

For a time-series relation, the baseline also produces recency and volume:

- `seconds_since_last`
- `num_events_{period}d`
- a safe short-window/long-window ratio for adjacent periods

3.1.2 Simple example

From the running event table, representative baseline values are:

Feature	Value	Meaning
<code>amount_sum</code>	60	Total amount over visible history
<code>amount_avg</code>	20	Mean amount per event
<code>status_nunique</code>	2	Two observed statuses
<code>status_invited_count</code>	2	Two invited events
<code>status_invited_share</code>	0.667	Two of three events were invited
<code>seconds_since_last</code>	86,400	Latest event was one day before cutoff
<code>num_events_7d</code>	2	Two events occurred in the 7-day window
<code>d1v7_change</code>	0.5	One 1-day event divided by two 7-day events

3.1.3 When to use it

Use it on nearly every reduced SQL child relation. It is cheap relative to the combinatorial families and establishes a strong relational baseline.

3.1.4 Important implementation detail

In the current SQL implementation, the conservative rollup loop is the baseline inside `sql_auto_features()` and the other family checks add operations around it. Treat the baseline as part of SQL auto-feature synthesis; omitting the literal string `"base"` is not a reliable way to suppress all ordinary type-aware rollups.

3.2 semantic: caller-defined meaning

3.2.1 What it does

`semantic` lets the caller name domain concepts with SQL expressions. It does not guess that a column named `status` means success or that an amount of 100 is high value. The caller supplies those meanings in `annotation_expressions`.

```
annotation_expressions={
  "is_invited": "{status} = 'invited'",
  "is_high_value": "{amount} >= 25",
  "weighted_amount": ("value", "{amount} * {quantity}"),
}
```

A plain expression is a predicate. GraphReduce compiles it to a numeric 0/1 column. A `("value", expression)` annotation preserves the expression's value. Column placeholders are resolved against the node's actual, possibly prefixed, columns.

3.2.2 Simple example

For `is_invited = status = 'invited'`, the three event rows are annotated as 1, 0, 1. Once reduced, ordinary numeric aggregation can produce:

Derived feature	Value
<code>is_invited_sum</code>	2
<code>is_invited_avg</code>	0.667
<code>is_invited_max</code>	1

If `conditional` is also enabled, the same predicate can be measured separately inside every lookback window. If `temporal` is enabled, a numeric value annotation can receive windowed sum/average/min/max features.

3.2.3 When to use it

Use it when a stable business rule is known and worth making explicit: a successful outcome, a severe incident, a premium transaction, a completed workflow, or a domain-specific amount.

Adding `"semantic"` without `annotation_expressions` does not invent domain features. Generic auto-annotation is a separate opt-in controlled by `auto_annotate_features=True` and its budget settings.

3.3 conditional: composition within time windows

3.3.1 What it does

`conditional` answers *what kind of activity occurred?* It selects conditions from explicit predicate annotations and sampled categorical values. Predicate annotations are prioritized. Free-form text, collections, dates, identifiers, and the reduction key are excluded.

For every selected condition and period it emits:

- `count`: rows satisfying the condition;
- `share`: condition count divided by all rows in the same window;
- `any`: whether the condition appeared at least once;
- `change`: short-window condition count divided by the next longer-window condition count.

3.3.2 Simple example

For the condition `status = 'invited'`:

Feature	Value	Explanation
<code>invited_count_7d</code>	1	Only the Jan 9 row is invited in 7 days
<code>invited_share_7d</code>	0.5	One of two 7-day events is invited
<code>invited_any_7d</code>	1	At least one invited event exists
<code>invited_d7v30_change</code>	0.5	One 7-day invite / two 30-day invites

3.3.3 When to use it

Use it for status mix, action type, channel, outcome, category, or a sparse business event where total event volume is too coarse.

3.3.4 Cost

With S selected conditions and P periods, the family adds approximately $S(4P - 1)$ columns. Keep `feature_family_max_columns` and `categorical_top_k` bounded, especially on deep graphs.

3.4 temporal: numeric magnitude by lookback window

3.4.1 What it does

`temporal` applies each configured lookback window to numeric measurements. Explicit numeric value annotations are selected first, followed by generic numeric columns, up to `feature_family_max_columns`. Identifiers, the reduction key, and the date key are excluded.

For every selected number and every period it emits windowed:

- sum;
- average;
- minimum;
- maximum.

The SQL uses conditional aggregates, so values outside the window become null inputs to the aggregate rather than leaking into the result.

3.4.2 Simple example

For `amount` in the running table:

Window	Sum	Average	Minimum	Maximum
1 day	30	30	30	30
7 days	50	25	20	30
30 days	60	20	10	30

Two users can have the same number of events but very different spending, duration, quantity, score, or severity. This family preserves that difference.

3.4.3 When to use it

Use it on time-series relations with meaningful numeric measurements. With N selected numeric columns and P periods, it adds approximately $4NP$ columns.

3.5 sequence: cadence, concentration, and active span

3.5.1 What it does

`sequence` describes the timing shape of activity without exposing an ordered event list. For every period it computes:

- `activity_rate_{period}d`: events in the window divided by window length;
- `activity_share_{period}d`: events in the window divided by lifetime events.

For adjacent windows it computes a burst ratio: short-window count divided by long-window count. Supported SQL dialects also receive:

- `active_span_seconds`: time from first visible event to last visible event;
- `activities_per_active_day`: lifetime event count divided by active span in days, with the implementation's one-day offset preventing zero division.

3.5.2 Simple example

For the three running events:

Feature	Value	Meaning
<code>activity_rate_7d</code>	0.286	2 events / 7 days
<code>activity_share_7d</code>	0.667	2 recent events / 3 lifetime events
<code>activity_burst_1v7</code>	0.5	1 event / 2 events
<code>active_span_seconds</code>	691,200	Eight days between Jan 1 and Jan 9
<code>activities_per_active_day</code>	0.333	3 events / (8 + 1) days

3.5.3 When to use it

Use it for churn, repeat behavior, engagement, burst detection, or time-to-event tasks. With P periods, it adds about $3P + 1$ columns when active-span date arithmetic is supported by the SQL dialect.

3.6 episode: row volume versus distinct records

3.6.1 What it does

`episode` always counts rows and, when a single primary key is available in the sample, also counts distinct primary keys. On time-series relations it repeats both counts inside every lookback window.

This distinction matters after a many-to-many join, where one logical record may appear in several rows.

3.6.2 Simple example

Imagine a joined order/product relation:

order_id	product
101	A
101	B
102	C

The episode features are:

Feature	Value	Interpretation
<code>num_episodes</code>	3	Three rows after the join
<code>num_unique_episodes</code>	2	Two distinct orders

The gap reveals multiplicity that a plain row count hides. With a usable single-column primary key, the family adds two lifetime columns and two columns per period. Composite primary keys currently skip the distinct-PK episode features and log a warning.

3.6.3 When to use it

Use it when join expansion, repeated records, or the distinction between line items and business events matters.

3.7 context: peer-relative row values

3.7.1 What it does

`context` compares a row with a caller-defined peer group *before* reduction. The caller must supply `context_keys`; GraphReduce deliberately does not guess whether `race_id`, `merchant_id`, `category`, or `event_id` is the meaningful peer group.

For each resolved context key it adds:

- peer-group row count (`context_size`);
- for selected numeric columns, the signed difference `value - peer_group_average(value)`.

The resulting row-level values then flow through the ordinary reduction, so a parent can receive summaries of its children's relative standing.

3.7.2 Simple example

Three drivers finish one race in positions 1, 4, and 2. Their race average is 2.333.

Driver position	Context size	Position minus race average
1	3	-1.333
4	3	1.667
2	3	-0.333

For race position, negative deltas are better than the peer average and positive deltas are worse. For price or score, the interpretation may reverse; the feature is signed, not inherently good or bad.

3.7.3 When to use it

Use it only when the peer group has a defensible relational or business meaning. Set both `feature_families` to include "context" and provide a non-empty `context_keys`. Numeric candidates per context key are capped by `feature_family_max_columns`.

4 How the families work together

The families are additive and often most useful in pairs:

Combination	What it captures
base + temporal	Lifetime magnitude plus recent numeric magnitude
semantic + conditional	Domain-specific event mix in each window
semantic + temporal	Domain-specific numeric value by window
base + sequence	Overall volume/recency plus cadence and burstiness
conditional + episode	Condition rates with explicit row/distinct-event denominators
context + base	Peer-relative row signals summarized to parent grain

Text deserves special mention: it is a baseline capability controlled by `auto_text_features`, not an eighth family. It measures shape and simple patterns; it does not tokenize text, create embeddings, or call a language model.

5 Configuration example

This node configuration enables all non-context families for the event table. The graph still needs `auto_features=True` and SQL execution.

```
import datetime
import duckdb
```

```

from graphreduce.enum import ComputeLayerEnum, PeriodUnit
from graphreduce.graph_reduce import GraphReduce
from graphreduce.node import DuckdbNode

# Assume the SQL tables `users` and `events` have been registered here.
connection = duckdb.connect()

users = DuckdbNode(
    fpath="users",
    fmt="sql",
    pk="user_id",
    prefix="usr",
    columns=["user_id"],
    compute_layer=ComputeLayerEnum.duckdb,
)

events = DuckdbNode(
    fpath="events",
    fmt="sql",
    pk="event_id",
    prefix="evt",
    date_key="event_time",
    columns=["event_id", "user_id", "event_time", "status", "amount"],
    compute_layer=ComputeLayerEnum.duckdb,
    feature_families=(
        "base",
        "semantic",
        "conditional",
        "temporal",
        "sequence",
        "episode",
    ),
    annotation_expressions={
        "is_invited": "{status} = 'invited'",
        "is_high_value": "{amount} >= 25",
    },
    ts_periods=(1, 7, 30),
    feature_family_max_columns=4,
    categorical_top_k=5,
)

graph = GraphReduce(
    name="user_features",
    parent_node=users,
    compute_layer=ComputeLayerEnum.duckdb,
    sql_client=connection,
    cut_date=datetime.datetime(2024, 1, 10),
    compute_period_val=30,
    compute_period_unit=PeriodUnit.day,
    auto_features=True,
    auto_labels=False,
)

graph.add_node(users)
graph.add_node(events)
graph.add_entity_edge(
    parent_node=users,

```



```

    relation_node=events,
    parent_key="user_id",
    relation_key="user_id",
    reduce=True,
)
graph.do_transformations_sql()

```

For context features, add a meaningful peer key on the relevant node:

```

results = DuckdbNode(
    fpath="results",
    prefix="res",
    pk="result_id",
    feature_families=("base", "context"),
    context_keys=("race_id",),
    feature_family_max_columns=4,
)

```

6 Time correctness and window semantics

Family selection does not replace temporal configuration. Correct historical features depend on:

- a valid `date_key` on every time-varying node;
- `cut_date`, which supplies the fixed point-in-time reference when no dynamic date node is present;
- `compute_period_val` and `compute_period_unit`, which bound the history that can enter the feature calculation;
- `ts_periods`, which define the nested day windows used by `base`, `conditional`, `temporal`, `sequence`, and `episode`.

The default `ts_periods` are 1, 3, 4, 7, 14, 30, 60, 90, 180, 365, and 730 days. Setting `ts_periods=()` disables the rolling and family-specific window features. A period longer than the compute horizon cannot recover older data: the input rows have already been filtered out. Raise the compute horizon when you need a genuinely longer history.

When a dynamic date node is available, windows can be evaluated relative to the current row's point-in-time reference. Otherwise they are relative to the graph cutoff. In both cases, validate a small sample at the exact window boundaries before a large run.

7 Budgeting and family selection

The most important controls are:

Setting	Controls
<code>feature_families</code>	Which named families are requested; unknown names fail fast
<code>ts_periods</code>	Number and length of lookback windows
<code>feature_family_max_columns</code>	Numeric candidates for <code>temporal/context</code> and conditions for <code>conditional</code>
<code>categorical_cardinality_threshold</code>	Whether all category values or a top-value subset is used by baseline summaries
<code>categorical_top_k</code>	Size of the high-cardinality category subset
<code>annotation_expressions</code>	Explicit semantic predicates and values
<code>auto_annotate_features</code>	Generic bounded annotations inferred from sampled data
<code>auto_text_features</code>	Baseline text-shape and pattern features
<code>context_keys</code>	Explicit peer groups for <code>context</code>

Approximate extra columns at one relation are useful for planning:

Family	Approximate count
Baseline time features	$2P$ (recency, P counts, $P - 1$ ratios)
<code>conditional</code>	$S(4P - 1)$
<code>temporal</code>	$4NP$
<code>sequence</code>	$3P + 1$ when span features are supported
<code>episode</code>	$2 + 2P$ with a single usable primary key; otherwise $1 + P$
<code>context</code>	Per peer key, one group-size annotation plus up to N numeric deltas before reduction

Here P is the number of periods, S the number of selected conditions, and N the number of selected numeric columns. These counts apply at one node and hop. Multiple child tables, repeated reductions, and downstream re-aggregation can multiply the final width and SQL work.

7.1 Recommended staged rollout

1. Start with the baseline and verify output grain, keys, and time boundaries.
2. Add `temporal` on numeric-heavy event tables.
3. Add `sequence` where cadence or recent concentration could matter.
4. Add `conditional` for a bounded set of statuses, categories, or predicates.
5. Add `episode` where row multiplicity is ambiguous.
6. Add explicit `semantic` rules that encode stable domain knowledge.
7. Add `context` only with a well-defined peer group.

8 Common pitfalls

Enabling families on the wrong backend. The named family program is SQL planner functionality. A pandas graph will still perform its own automatic aggregations, but it will not produce the same seven-family output.

Assuming a window expands the available history. `ts_periods=(365,)` cannot see a year of data if the compute horizon is only 90 days.

Using a category as free-form text, or text as a category. The inference sample controls this decision. Inspect representative samples and keep high-cardinality settings bounded.

Confusing rows with business events. Many-to-many joins can duplicate a primary key. Use `episode` and inspect `num_episodes` versus `num_unique_episodes`.

Choosing a plausible but wrong context key. Peer-relative features can look reasonable even when the comparison group has no causal or business meaning. GraphReduce requires `context_keys` precisely because this choice needs human judgment.

Generating every family everywhere. Family width compounds across graph hops. Put specialized families on nodes where their input columns and intended signal are meaningful.

9 Quick decision guide

Choose the first matching question:

- Need a reliable general-purpose relational baseline? Use `base`.
- Know a business predicate or derived value? Add `semantic`.
- Care about the recent mix of statuses or event types? Add `conditional`.
- Care about recent totals or ranges of numeric values? Add `temporal`.
- Care about regularity, bursts, or active duration? Add `sequence`.
- Care whether several rows represent one underlying record? Add `episode`.
- Care how each row compares with a meaningful peer group? Add `context`.

Most production configurations should be selective rather than maximal. The best family is the one whose output has a clear interpretation at the node, time window, and final model grain.

10 Source basis

This report was checked against the local GraphReduce 1.10.1 source at commit 5600d1d, particularly `graphreduce/node.py`, the SQL feature-family tests in `tests/test_graph_reduce.py`, and the feature-family documentation in `README.md`. The report describes the behavior present in that local source snapshot; later releases may change defaults, supported dialects, or feature naming.